# Избор инструкција

# Поједностављени поглед на задњи део компајлера

Међурепрезентација
(Међујезик – IR)




Инструкције циљне архитектуре

# Поједностављени поглед на задњи део компајлера

Међурепрезентација
(Међујезик – IR)




Инструкције циљне архитектуре

# Поједностављени поглед на задњи део компајлера

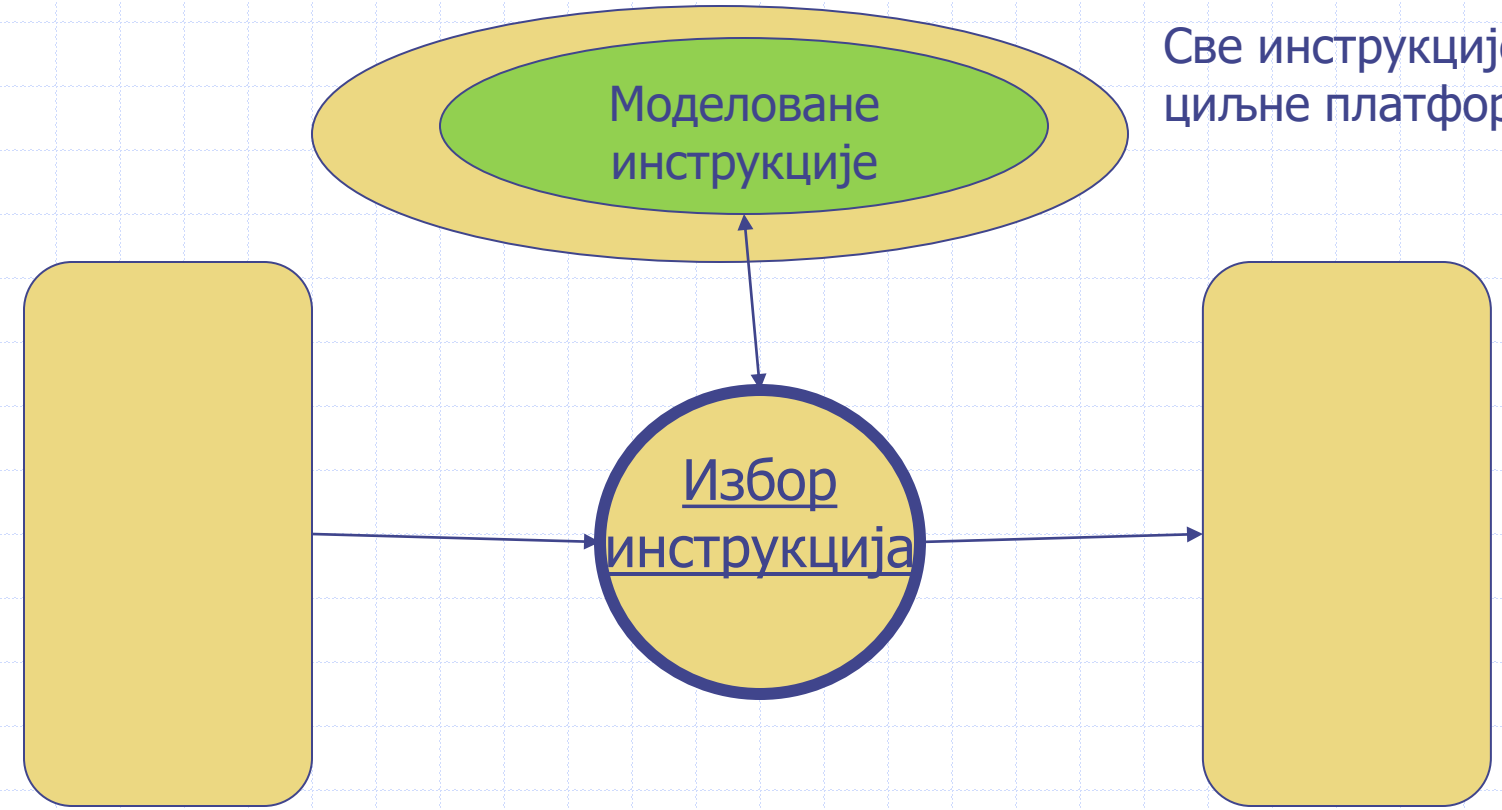
Међурепрезентација
(Међујезик – IR)




Да ли Избор инструкција мора бити прва фаза?

Псеудо-асемблер
(листа инструкција које још увек не баратају стварним ресурсима)

# Ближи погледа на избор инструкција



# Задаци избора инструкција

- Ваљаност
  - Изабране инструкције морају бити семантички еквивалентне програму у међукоду
- Квалитет
  - Изабрани скуп инструкција треба да буде што бољи, у складу са неким задатим критеријумом

Исте задатке има и компајлер као целина.

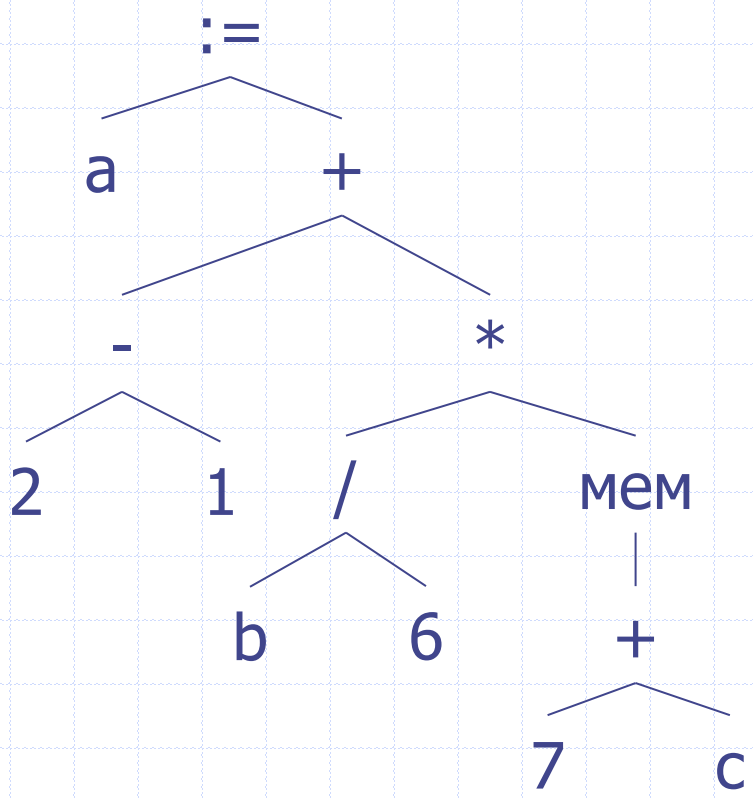
# Разни поступци за избор инструкција

- Поступци се разликују у одређеним елементима у зависности од тога каква је форма међурепрезентације над којом могу да раде
- У основи сваког поступка су **правила** која упарују део МР са неким скупом инструкција
- Правила могу бити „1 на 1“, „1 на више“, „више на 1/више“.
- Разликују се многе друге особине: врсте инструкција које се могу моделовати у правилима, поступак избора правила и сл.

# Пример једноставне МР

:=
+
-
*
/
мем

var
const



# Правила „1 на 1“

:=
+
-
*
/
мем

var
const

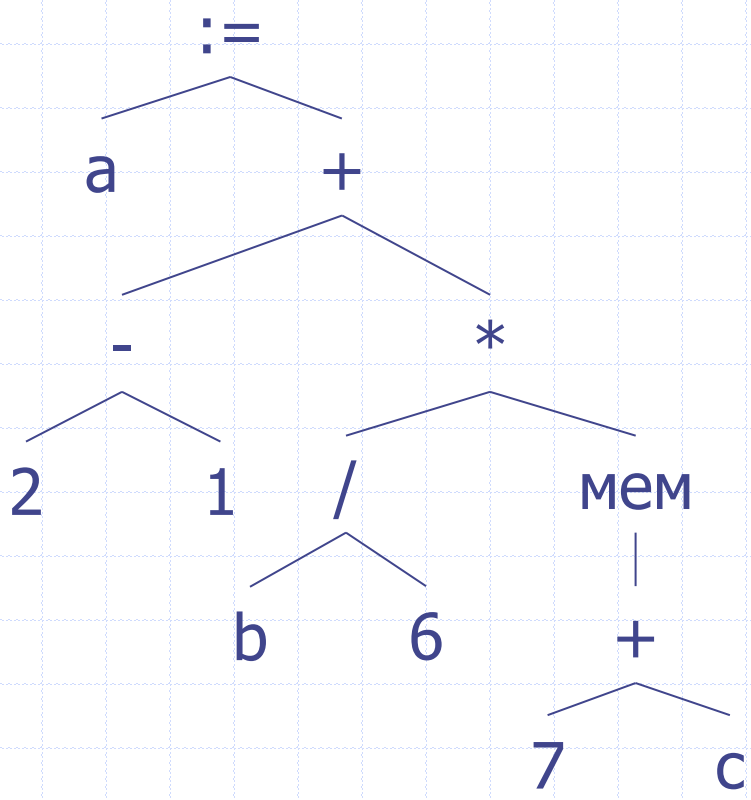
| | | |
|---|---|---|
| := | -> | mov |
| + | -> | add |
| - | -> | sub |
| * | -> | mul |
| / | -> | div |
| мем | -> | ld |
| const | -> | ldi |

mov $r_d$, $r_{s1}$, $r_{s2}$
add $r_d$, $r_{s1}$, $r_{s2}$
sub $r_d$, $r_{s1}$, $r_{s2}$
mul $r_d$, $r_{s1}$, $r_{s2}$
div $r_d$, $r_{s1}$, $r_{s2}$
ld $r_d$, $r_s$
ldi $r_d$, imm

var -> претпоставка да је већ у регистру

# Правила „1 на 1“

| | | |
|---|---|---|
| := | -> | mov |
| + | -> | add |
| - | -> | sub |
| * | -> | mul |
| / | -> | div |
| мем | -> | ld |
| const | -> | ldi |



# Избор инструкција

```
? setnja(Node* x) {
  switch (x->getKind()) {
    case MOV_OK:
      MovNode* p = static_cast<MovNode*>(x);
      Reg lop = setnja(p->leftNode());
      Reg rop = setnja(p->rightNode());
      emitInstr(MovInstr(lop, rop));
      return ?;
    break;
    case ADD_OK: ...
    case SUB_OK: ...
    case MUL_OK: ...
    case DIV_OK: ...
    case MEM_OK: ...
    case CONST_OK: ...
    case VAR_OK: ...
  }
}
```

# Избор инструкција

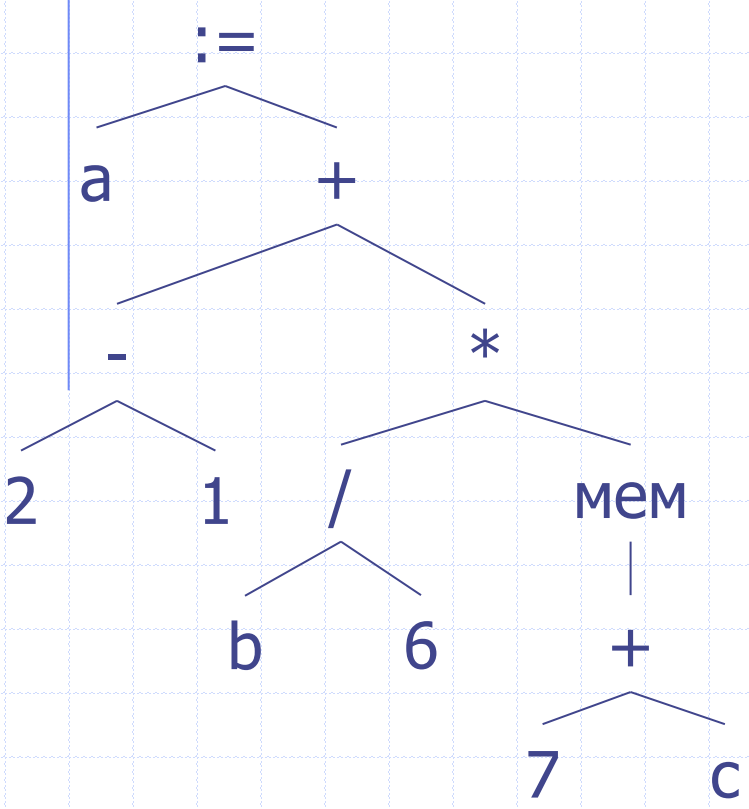
```
void setnjaStm(Node* x) {
  switch (x->getKind()) {
    case MOV_OK:
      MovNode* p = static_cast<MovNode*>(x);
      Reg lop = setnjaExp(p->leftNode());
      Reg rop = setnjaExp(p->rightNode());
      emitInstr(MovInstr(lop, rop));
      return;
    ...
}

Reg setnjaExp(Node* x) {
  switch (x->getKind()) {
    case ADD_OK: ...
    ...
  }
}
```

# Избор инструкција

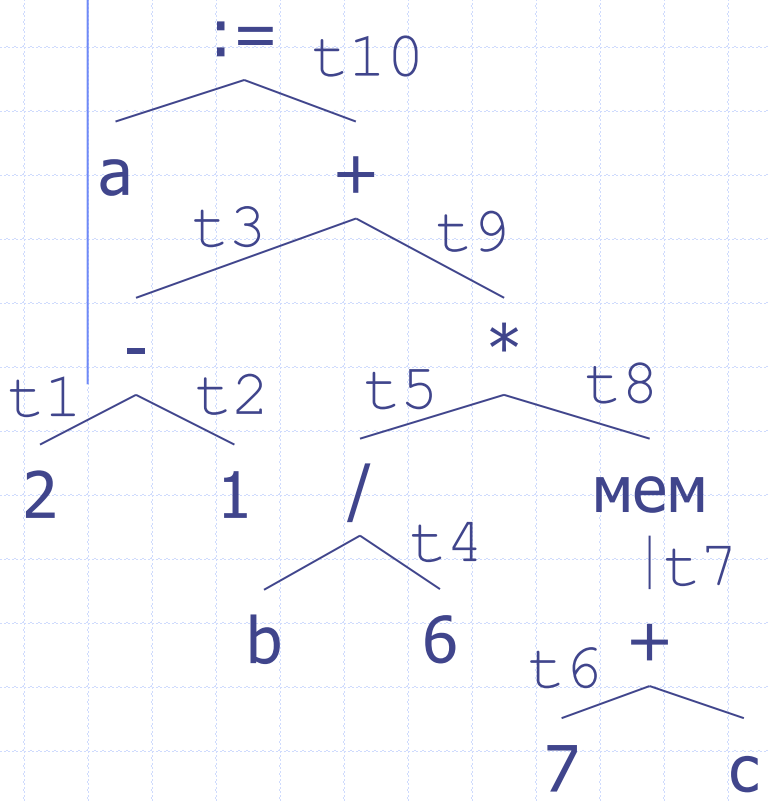
```
Reg setnjaExp(Node* x) {
  switch (x->getKind()) {
    case ADD_OK:
      AddNode* p = static_cast<AddNode*>(x);
      Reg lop = setnjaExp(p->leftNode());
      Reg rop = setnjaExp(p->rightNode());
      Reg dst = NewTemp();
      emitInstr(AddInstr(dst, lop, rop));
      return dst;
    case SUB_OK: ...
    case MUL_OK: ...
    case DIV_OK: ...
    case MEM_OK: ...
    case CONST_OK: ...
    case VAR_OK:
      VarNode* p = static_cast<VarNode*>(x);
      return p->getReg();
  }
}
```

# Избор инструкција




```
ldi t1, 2
ldi t2, 1
sub t3, t1, t2
ldi t4, 6
div t5, b, t4
ldi t6, 7
add t7, t6, c
ld  t8, t7
mul t9, t5, t8
add t10, t3, t9
mov a, t10
```

# Избор инструкција




```
ldi t1, 2
ldi t2, 1
sub t3, t1, t2
ldi t4, 6
div t5, b, t4
ldi t6, 7
add t7, t6, c
ld  t8, t7
mul t9, t5, t8
add t10, t3, t9
mov a, t10
```

# МР у форми листе

```
t1 <- 2
t2 <- 1
t3 <- t1 - t2
t4 <- 6
t5 <- b - t4
t6 <- 7
t7 <- t6 + c
t8 <- мем[t7]
t9 <- t5 * t8
t10 <- t3 + t9
a := t10
```

# Избор инструкција - MP у форми листе

```
void izbor(IR x) {
  for (auto& it : x.opList()) {
    switch (it.kind) {
      case ADD_OK:
        emitInstr(AddInstr(it.dst, it.src1, it.src2));
        break;
      case SUB_OK: ... break;
      case MUL_OK: ... break;
      case DIV_OK: ... break;
      case MEM_OK: ... break;
      case CONST_OK: ... break;
      //case VAR_OK: ...
    }
  }
}
```

# Форме међурепрезентације

- Структуралне:
  - Стабла, усмерени ациклични графови, графови
- Линеарне:
  - Листа инструкција апстрактне машине
  - Тро-адресни код, код стек машине
- Хибридне (комбинација претходна два):
  - Граф тока управљања

# Правила – пример „1 на више“

:=
+
-
*
/
мем

var
const

| | | |
|---|---|---|
| := | -> | mov |
| + | -> | add |
| - | -> | inv<br>add |
| * | -> | mul |
| / | -> | div |
| мем | -> | ld |
| const | -> | ldi |

mov $r_d$, $r_{s1}$, $r_{s2}$
add $r_d$, $r_{s1}$, $r_{s2}$
**inv $r_d$, $r_s$**
mul $r_d$, $r_{s1}$, $r_{s2}$
div $r_d$, $r_{s1}$, $r_{s2}$
ld $r_d$, $r_s$
ldi $r_d$, imm

var -> претпоставка да је већ у регистру

# Избор инструкција – пример „1 на више“

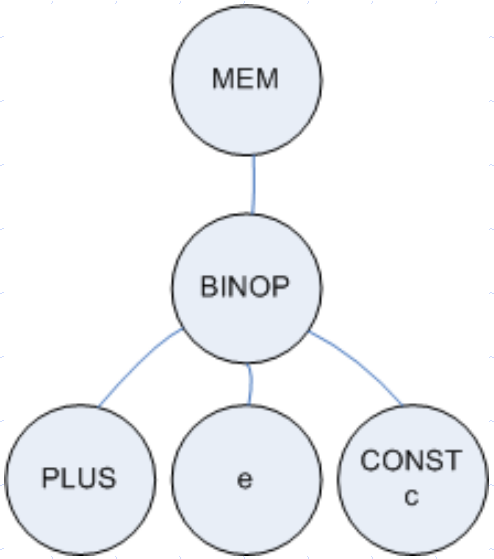
```
Reg setnjaExp(Node* x) {
  switch (x->getKind()) {
    case ADD_OK: ...
    case SUB_OK:
      SubNode* p = (SubNode*)x;
      Reg rop = setnjaExp(p->rightNode());
      Reg lop = setnjaExp(p->leftNode());
      Reg tmpd = NewTemp();
      emitInstr(InvInstr(tmpd, rop));
      Reg dst = NewTemp();
      emitInstr(AddInstr(dst, lop, tmpd));
      return dst;
    case MUL_OK: ...
    case DIV_OK: ...
    case MEM_OK: ...
    case CONST_OK: ...
    case VAR_OK: ...
    }
}
```

# Избор инструкција – пример „1 на више“

- МР у форми листе?

# Шаблони стабла – Правила „више на више“

- IR чворови изражавају само по једну операцију
  - Читање или упис у меморију, сабирање или одузимање...
- Реална инструкција може обавити више операција
  - Нпр. скоро свака машина може обавити сабирање и дохватање операнда у истој инструкцији
    - Модел инструкције у IR = шаблон стабла



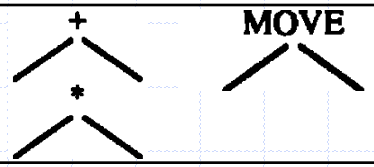
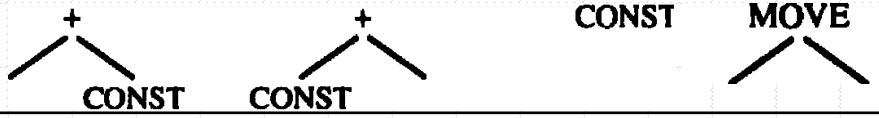
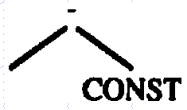
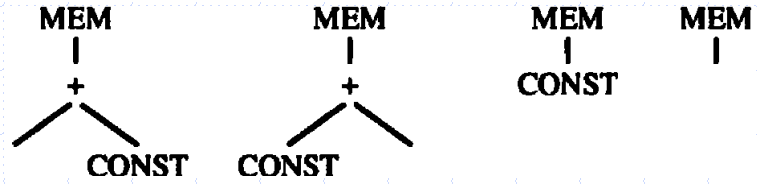
# Оптималност избора инструкција

- Постојање правила „више на *“ уводи тему оптималности избора инструкција, јер може постојати више различитих покривања стабла.
- Оптималност над истим скупом моделованих инструкција.
  - Шта ако неке важне инструкције нису моделоване?
- Оптималност над истим скупом правила.
  - Како се то односи према укупној оптималности компајлера?
- Оптималност?
  - По ком критеријуму?

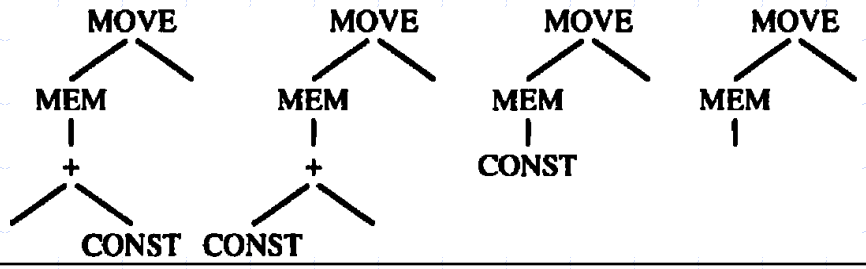
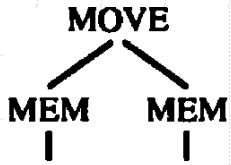
# Пример скупа инструкција (1/3)
## Jouette архитектура

- **Регистар r0 увек садржи 0**
- M[x] означава мем. локацију са адресом x
- Инструкције
  - Аритметичке
    - Производе резултате у регистру
  - Меморијске
    - Изводе ивичне ефекте над меморијом (пишу у меморију)
- Неким инструкцијама одговара више шаблона
  - комутативни оператори (+ и *)
  - LOAD/STORE: регистар или константа може бити 0

# Пример скупа инструкција (2/3)
## Jouette архитектура

| Име | Ефекат | Стабла |
|---|---|---|
| | $r_i$ | TEMP |
| ADD<br>MUL | $r_i \leftarrow r_j + r_k$<br>$r_i \leftarrow r_j \times r_k$ |  |
| SUB<br>DIV | $r_i \leftarrow r_j - r_k$<br>$r_i \leftarrow r_j / r_k$ |  |
| ADDI | $r_i \leftarrow r_j + c$ |  |
| SUBI | $r_i \leftarrow r_j - c$ |  |
| LOAD | $r_i \leftarrow M[r_j + c]$ |  |

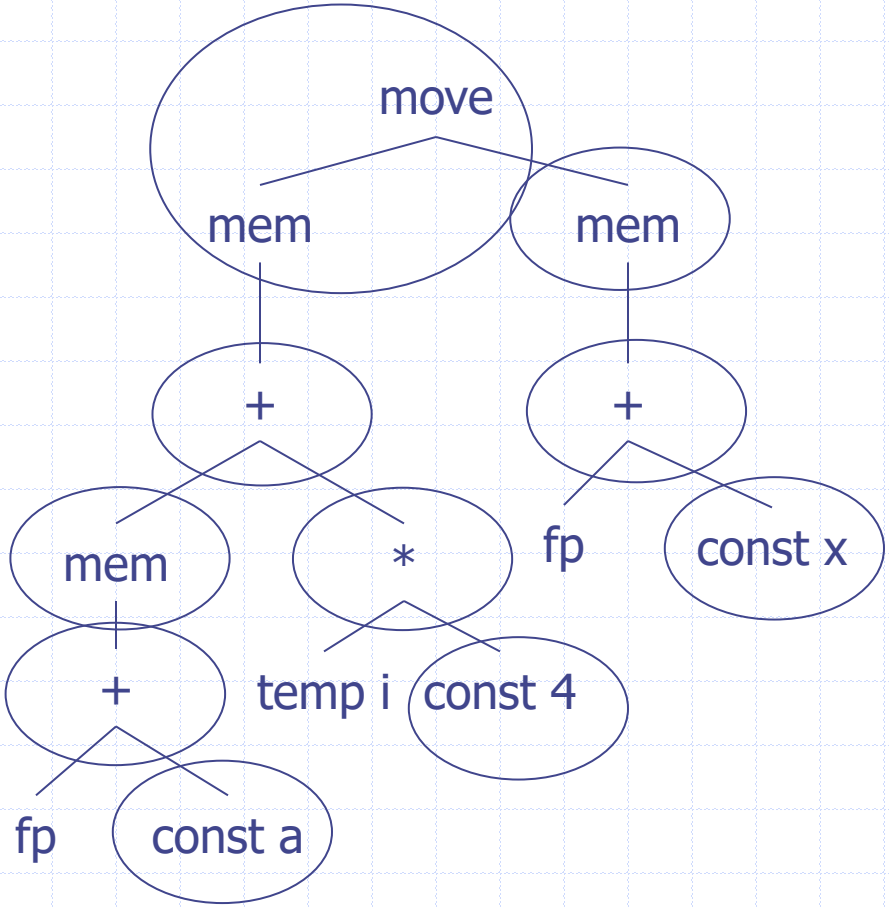
# Пример скупа инструкција (3/3)
## Jouette архитектура

| | |
|---|---|
| **STORE** $M[r_j + c] \leftarrow r_i$ |   |
| **MOVEM** $M[r_j] \leftarrow M[r_i]$ |   |

# Пример поплочавања

## Тајгер исказ a[i] := x

| | |
|---|---|
| ADDI | $r_1 \leftarrow r_0 + a$ |
| ADD | $r_1 \leftarrow fp + r_1$ |
| LOAD | $r_1 \leftarrow M[r_1 + 0]$ |
| ADDI | $r_2 \leftarrow r_0 + 4$ |
| MUL | $r_2 \leftarrow r_i \times r_2$ |
| ADD | $r_1 \leftarrow r_1 + r_2$ |
| ADDI | $r_2 \leftarrow r_0 + x$ |
| ADD | $r_2 \leftarrow fp + r_2$ |
| LOAD | $r_2 \leftarrow M[r_2 + 0]$ |
| STORE | $M[r_1 + 0] \leftarrow r_2$ |

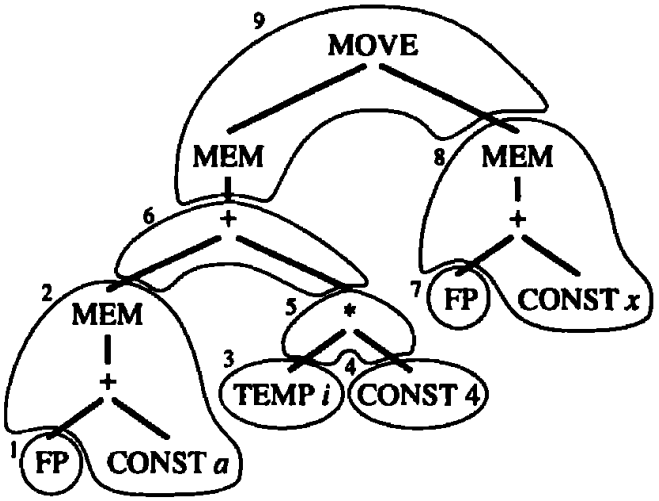


# Пример поплочавања

## Тајгер исказ a[i] := x

```
ADDI    r1 ← r0 + a
ADD     r1 ← fp + r1
LOAD    r1 ← M[r1 + 0]
ADDI    r2 ← r0 + 4
MUL     r2 ← ri x r2
ADD     r1 ← r1 + r2
ADDI    r2 ← r0 + x
ADD     r2 ← fp + r2
LOAD    r2 ← M[r2 + 0]
STORE   M[r1 + 0] ← r2
```

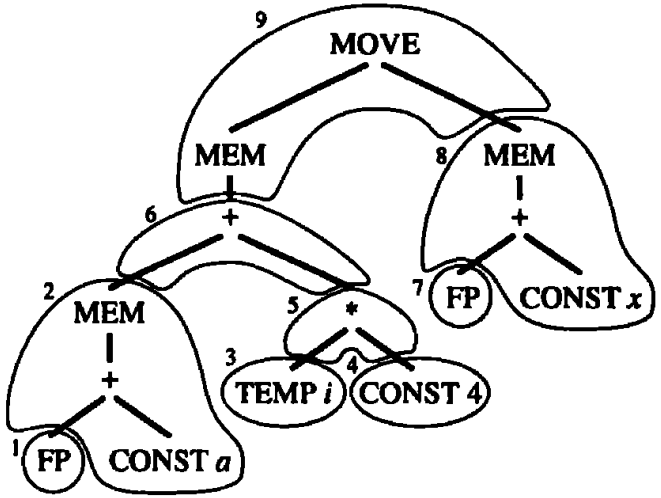
```
LOAD    r1 ← M[fp + a]
ADDI    r2 ← r0 + 4
MUL     r2 ← ri x r2
ADD     r1 ← r1 + r2
LOAD    r2 ← M[fp + x]
STORE   M[r1 + 0] ← r2
```



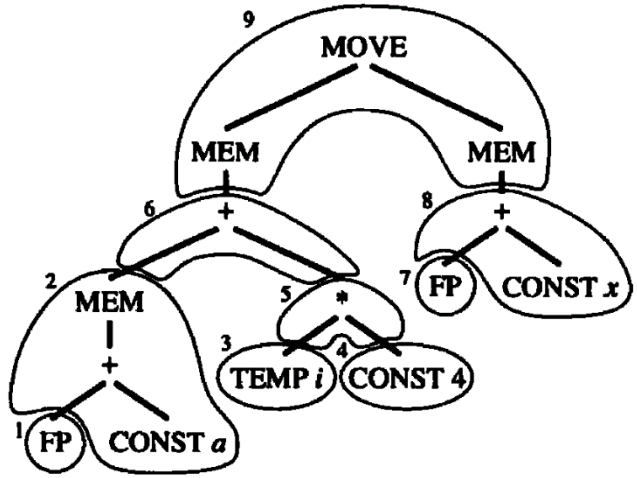

Које поплочавање је боље?

# Пример поплочавања

## Тајгер исказ a[i] := x




| | | |
|---|---|---|
| 2 | LOAD | $r_1 \leftarrow M[fp + a]$ |
| 4 | ADDI | $r_2 \leftarrow r_0 + 4$ |
| 5 | MUL | $r_2 \leftarrow r_i \times r_2$ |
| 6 | ADD | $r_1 \leftarrow r_1 + r_2$ |
| 8 | LOAD | $r_2 \leftarrow M[fp + x]$ |
| 9 | STORE | $M[r_1 + 0] \leftarrow r_2$ |




| | | |
|---|---|---|
| 2 | LOAD | $r_1 \leftarrow M[fp + a]$ |
| 4 | ADDI | $r_2 \leftarrow r_0 + 4$ |
| 5 | MUL | $r_2 \leftarrow r_i \times r_2$ |
| 6 | ADD | $r_1 \leftarrow r_1 + r_2$ |
| 8 | ADDI | $r_2 \leftarrow fp + x$ |
| 9 | MOVEM | $M[r_1] \leftarrow M[r_2]$ |

Које поплочавање је боље?

# Пример поплочавања

## Тајгер исказ a[i] := x

- У исказу a[i] := x, *i* је у регистру, док су *a* и *x* у оквиру стека
- Показана су два могућа поплочавања
  - *a* и *x* су у ствари одстојања од показивача оквира, fp
  - У оба случаја, плочице 1, 3 и 7 не одговарају инструкцијама, већ регистрима (TEMP-ови)
- Иначе, увек је могуће IR поплочати танким плочицама, које покривају по један IR чвор

# Алгоритам максималног залогаја (енгл. munch)

- је алгоритам субоптималног поплочавања IR
- Врло је једноставан
  - Полазећи од корена IR пронаћи највећи шаблон који се уклапа и њиме покрити корен и чворове испод њега
  - Поновити исти алгоритам за сва преостала подстабла
- Инструкције се генеришу у обратном редоследу
  - Инструкција у корену је прва генерисана,
    - али може да се изврши тек кад све остале инструкције произведу њене операнде

# Појашњење/изведба алгоритма

- Највећи шаблон је онај са највише IR чворова
- Ако се два шаблона исте величине уклапају на неком месту, избор између њих је произвољан
- Изведба за Jouette: две рекурзивне функције
  - munchStm за исказе и munchExp за изразе
  - Клаузуле munchExp-а одговарају шаблонима
    - Клаузуле су поређане по опадајућој величини шаблона
- Следе скелети функција
  - Изостављен је избор регистара и емитовање операнада

# Костур функције munchExp

```
Temp munchExp(T_exp exp)
  MEM(BINOP(PLUS, e1, CONST(i))) => munchExp(e1); emit("LOAD");
  MEM(BINOP(PLUS, CONST(i), e1)) => munchExp(e1); emit("LOAD");
  MEM(CONST(i)) => emit("LOAD");
  MEM(e1) => munchExp(e1); emit("LOAD");
  BINOP(PLUS, e1, CONST(i)) => munchExp(e1); emit("ADDI");
  BINOP(PLUS, CONST(i), e1) => munchExp(e1); emit("ADDI");
  CONST(i) => emit("ADDI");
  BINOP(PLUS, e1, e2) => munchExp(e1); munchExp(e2); emit("ADD");
  TEMP(t) => {}
```

# Костур функције munchStm

```
void munchStm(T_stm stm)
  MOVE(BINOP(PLUS, e1, CONST(i)), e2)
    => munchExp(e1); munchExp(e2); emit("STORE");
  MOVE(BINOP(PLUS, CONST(i), e1), e2)
    => munchExp(e1); munchExp(e2); emit("STORE");
  MOVE(MEM(e1), MEM(e2))
    => munchExp(e1); munchExp(e2); emit("MOVE");
  MOVE(MEM(CONST(i)), e2) => munchExp(e2); emit("STORE");
  MOVE(MEM(e1), e2) => munchExp(e1); munchExp(e2); emit("STORE");
  MOVE(TEMP(i), e2) => munchExp(e2); emit("ADD");
  LABEL(lab) => makeLabel(lab);
```

# Брзо поклапање (1/2)

- Алгоритми поплочавања морају за сваки чвор IR
  - Проверити све шаблоне, који се могу уклопити у чвору
- Наиван алгоритам би редом проверавао сваки шаблон и сваки чвор шаблона са одговарајућим делом IR
- Бољи приступ: да би се уклопио шаблон у чвору n
  - Лабела чвора n се користи за избор case клаузуле

```
match(n) {
  switch (label(n))  {
    case MEM:   …
    case BINOP:   …
    case CONST:   …
…
```

# Брзо поклапање (2/2)

- Једном кад се изабере једна лабела, нпр. MEM
  - Даље се анализирају само шаблони који имају ту лабелу у свом корену
    - Нови switch-case
- Правило: низ поређења у munchExp никада не треба да гледа исти чвор IR два пута

# Костур функције munchStm (1/2)

```
void munchStm(T_stm s) {
  switch (s->kind) {
    case T_MOVE: {
      T_exp dst = s->u.MOVE.dst, src = s->u.MOVE.src;
      if (dst->kind == T_MEM) {
        if (dst->u.MEM->kind == T_BINOP
          && dst->u.MEM->u.BINOP.op == T_plus
          && dst->u.MEM->u.BINOP.right->kind == T_CONST) {
            // MOVE(MEM(BINOP(PLUS, e1, CONST(i))), e2)
            T_exp e1 = dst->u.MEM->u.BINOP.left, e2 = src;
            munchExp(e1); munchExp(e2); emit("STORE");
        } else if (dst->u.MEM->kind == T_BINOP
            && dst->u.MEM->u.BINOP.op == T_plus
            && dst->u.MEM->u.BINOP.left->kind == T_CONST) {
              // MOVE(MEM(BINOP(PLUS, CONST(i), e1)), e2)
              T_exp e1 = dst->u.MEM->u.BINOP.right, e2 = src;
              munchExp(e1); munchExp(e2); emit("STORE");
```

# Костур функције munchStm (2/2)

```
    } else if (src->u.MEM->kind == T_MEM) {
      // MOVE(MEM(e1), MEM(e2))
      T_exp e1 = dst->u.MEM, e2 = src->u.MEM;
      munchExp(e1); munchExp(e2); emit("MOVEM");
    } else {
      // MOVE(MEM(e1), e2)
      T_exp e1 = dst->u.MEM, e2 = src;
      munchExp(e1); munchExp(e2); emit("STORE");
  } else if (dst->kind == T_TEMP) {
    // MOVE(TEMPi, e2)
    T_exp e2 = src;
    munchExp(e2); emit("ADD"); //second reg is r0 (==0)
  } else assert(0); //destination of MOVE must be MEM or TEMP
  break;
case T_JUMP: ...
case T_CJUMP: ...
case T_NAME: ...
...
```

# Избор инструкција за Тајгер

- Апстрактни асемблер
- Мапе привремених променљивих, temp
- Пример генерисаних асемблерских инструкција
- Костури MunchStm и MunchExp
- Позиви процедура и функција
- Излаз

# Апстрактни асемблер

```
enum InstrKind {I_OPER, I_LABEL, I_MOVE};

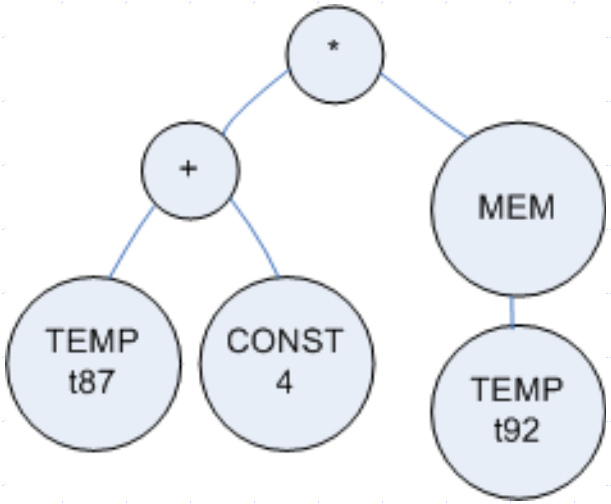
Instr* makeOper(string a, TempList d, TempList s);
Instr* makeLabel(string a, Label label);
Instr* makeMove(string a, TempList d, TempList s);

void printInst(FILE* out, const Intr& i, TempMap m);
```

# Пример генерисаних инструкција

- Дато IR стабло би могло бити преведено у следеће Jouette инструкције
  - ‘si/‘di означавају i-ти изворишни/одредишни регистар
    - Неки пут се регистар подразумева и не наводи се експлицитно
    - Нпр. за инструкцију add t1,t2, која реализује t1←t1+t2, генерише се стринг add ‘d0,‘s1; регистар ‘s0 је имплицитан

| Asem | dst | src |
|---|---|---|
| ADDI ‘d0 <- ‘s0+4 | t98 | t87 |
| LOAD ‘d0 <- M[‘s0+0] | t99 | t92 |
| MUL ‘d0 <- ‘s0*‘s1 | t100 | t98,t99 |



# munchExp (1/2)

```
Temp_temp munchExp(T_exp e) {
  switch (e) {
    case MEM(BINOP(PLUS, e1, CONST(i))): {
      Temp r = newTemp ();
      emit(makeOper("LOAD 'd0 <- M['s0+"+ i + "]\n",
                    {r}, {munchExp(e1)}));
      return r;}
    case MEM(CONST(i)): {
      Temp r = newTemp();
      emit(makeOper("LOAD 'd0 <- M[r0+" + i + "]\n",
                     {r}, {}));
      return r;}
    case MEM(e1): {
      Temp r = newTemp ();
      emit(makeOper("LOAD 'd0 <- M['s0+0]\n",
                    {r}, {munchExp(e1)}));
      return r;}
```

# munchExp (2/2)

```
case BINOP(PLUS, CONST(i), e1): {
  Temp r = newTemp ();
  emit(makeOper("ADDI 'd0 <- 's0+" + i + "]\n",
                {r}, {munchExp(e1)}));
  return r;}
case CONST(i): {
  Temp r = newTemp ();
  emit(makeOper("ADDI 'd0 <- r0+" + i + "]\n",
                {r}, {}));
  return r;}
case BINOP(PLUS, e1, e2): {
  Temp r = newTemp ();
  emit(makeOper("ADD 'd0 <- 's0+'s1\n",
    {r}, {munchExp(e1), munchExp(e2)}));
  return r;}
case TEMP(t):
  return t;
```

...

# munchStm (1/2)

```
void munchStm(T_stm s) {
  switch (s) {
    case MOVE(BINOP(PLUS, e1, CONST(i)), e2):
      emit(makeOper("STORE M[`s0+" + i + "] <- `s1\n",
        {}, {munchExp(e1), munchExp(e2)}));
    case MOVE(BINOP(PLUS, CONST(i), e1), e2):
      emit(makeOper("STORE M[`s0+" + i + "] <- `s1\n",
        {}, {munchExp(e1), munchExp(e2)}));
    case MOVE(MEM(e1), MEM(e2)):
      emit(makeOper("MOVE M[`s0] <- M[`s1]\n",
        {}, {munchExp(e1), munchExp(e2)}));
    case MOVE(MEM(CONST(i)), e2):
      emit(makeOper("STORE M[r0+" + i + "] <- 's0\n",
        {}, {munchExp(e2)}));
```

# munchStm (2/2)

```
    case MOVE(MEM(e1), e2):
      emit(makeOper("STORE M['s0] <- 's1\n",
        {}, {munchExp(e1), munchExp(e2)}));
    case MOVE(TEMP(i), e2):
      emit(makeMove("ADD 'd0 <- 's0 +r0\n",
        {i}, {munchExp(e2)}));
    case LABEL(lab):
      emit(makeLabel(toString(lab) + ":\n"s, lab);
...
```

# Пример генерисаног псеудо кода
## queens, процедура printboard

```
PROCEDURE        printboard
...
ADD 184 <- $fp + r0
ADDI 185 <- r0+1
ADD 132 <- 185 + r0
LOAD 186 <- M[$fp+12]
ADDI 190 <- r0+4
LOAD 191 <- M[$fp+4]
MUL 189 <- 191 * 190
LOAD 193 <- M[$fp+0]
LOAD 192 <- M[193+20]
ADD 188 <- 192 + 189
LOAD 187 <- M[188+0]
CMP 187,186
JEQ L91
L92:
ADDI 194 <- r0+0
ADD 132 <- 194 + r0
L91:
ADDI 195 <- r0+1
...
```

```
function printboard() =
  (for i := 0 to N-1
    do (for j := 0 to N-1
      do print(if col[i]=j then " O"
                           else " .");
         print("\n"));
     print("\n"))
```